

Timing + Acoustic Keyboard Recognition

Step-by-Step Implementation Plan

A simple beginning-to-end plan for building a consent-based research prototype that compares acoustic-only, timing-only, and fused keyboard recognition models.

Core question	Does inter-keystroke timing improve recovery compared with acoustics alone?
First scope	One keyboard, one microphone setup, controlled prompts, exact key timestamps.
Main comparison	Acoustic only vs. timing only vs. acoustic + timing.
Research framing	Measure leakage, evaluate limitations, and test defenses.

Project Goal

Build a controlled, consent-based research prototype that measures whether keyboard timing and rhythm improve acoustic key or text recovery.

The central research question is:

Does inter-keystroke timing improve end-to-end key or text recovery compared with acoustics alone?

This project should be treated as a security and privacy research project. Use only consenting participants, your own equipment, and synthetic prompts. Do not collect real passwords or private text.

Simple System Overview

The full system has five major stages:

```
audio recording
-> keystroke segmentation
-> per-keystroke acoustic classifier
-> timing/rhythm feature extractor
-> sequence decoder
-> final key or text hypothesis
```

The acoustic model gives local evidence:

```
keypress 17 sounds like:
k: 0.42
j: 0.31
h: 0.08
u: 0.05
...
```

The timing model gives sequence evidence:

```
the interval between keypress 17 and 18 is more consistent with:
- nearby keys
- alternating hands
- a common digraph
- a pause or word boundary
```

The fusion model combines both kinds of evidence.

Phase 0: Define Scope

Start with a narrow, controlled version.

Initial scope:

- one keyboard or laptop
- one microphone setup
- one quiet room
- controlled typing app
- known prompts

- audio plus exact keydown and keyup timestamps
- consenting participants only

The first version should not try to handle every keyboard, every user, every room, and every microphone. The first goal is to prove whether timing adds measurable value.

The main comparison should be:

```
acoustic only
vs.
timing only
vs.
acoustic + timing
```

Phase 1: Build The Data Collection Tool

Create a small local typing app. A web app is enough.

The app should:

- show a prompt to type
- record keydown timestamps
- record keyup timestamps
- record actual typed characters
- save a trial ID
- save participant ID
- save metadata about keyboard, microphone, room, and date
- coordinate with audio recording

Each trial should save:

```
trial_id
participant_id
prompt_text
typed_text
keydown_time
keyup_time
key_code
character
audio_file_path
keyboard_model
microphone_model
mic_position
recording_sample_rate
room_notes
```

Use the same machine for typing timestamps and audio recording if possible. That makes alignment much easier.

Phase 2: Design Typing Tasks

Collect both isolated-key data and natural typing data.

Use four prompt types:

- A. isolated keys
- B. common English phrases
- C. random character sequences
- D. synthetic password-like strings

Why each type matters:

- isolated keys help train the acoustic classifier
- English phrases capture natural typing rhythm
- random strings test whether language statistics are doing the work
- synthetic password-like strings test secret-entry behavior without collecting real secrets

Example isolated prompts:

```
aaaaa  
bbbbb  
jjjjj  
kkkkk  
99999
```

Example phrase prompts:

```
the quick brown fox  
we need to meet after lunch  
security research requires care
```

Example random prompts:

```
g7qk2m  
x9pa4z  
lq0vnt
```

Example synthetic password-like prompts:

```
River-742  
Mango!39  
T7_blue_hat
```

Suggested pilot dataset:

```
5 users  
1 keyboard  
100 trials per user
```

Suggested expanded dataset:

```
10 to 20 users  
2 to 3 keyboard types  
multiple microphone positions  
multiple sessions
```

Phase 3: Record Audio Carefully

Use a consistent audio setup for the first version.

Recommended baseline:

```
WAV format  
44.1 kHz or 48 kHz  
16-bit or 24-bit
```

```
fixed microphone location
quiet room
```

Record one continuous audio clip per trial.

For each session, record metadata:

- microphone model
- microphone distance from keyboard
- microphone angle
- keyboard or laptop model
- desk surface
- room noise notes
- participant posture notes
- sample rate and bit depth

Optional but useful: play a short beep at the start of each trial and log its timestamp. The beep creates an audio marker that helps align the audio timeline with the keyboard event timeline.

Phase 4: Preprocess Audio

For every trial, isolate each keystroke from the full audio clip.

Start with oracle segmentation, which uses the ground-truth keydown timestamp:

```
for each keydown timestamp:
    extract audio from keydown_time - 50 ms
    to keydown_time + 250 ms
```

This avoids confusing segmentation errors with classification errors.

Later, add automatic segmentation:

```
audio recording
-> compute short-time energy
-> detect peaks
-> match peaks to key events
-> extract each keystroke clip
```

Keep both evaluation modes:

- oracle segmentation: uses true timestamps
- automatic segmentation: uses audio only

This lets you measure how much performance is lost when segmentation is imperfect.

Phase 5: Build The Acoustic Model

The acoustic model predicts a key from one isolated sound.

Basic pipeline:

```
keystroke audio clip
```

```
-> normalize volume
-> convert to log-mel spectrogram
-> acoustic classifier
-> probability over keys
```

Start with log-mel spectrograms:

```
64 mel bands
1024 FFT window
fixed input duration
fixed-size spectrogram image
```

Possible model choices:

- small CNN
- ResNet-style CNN
- CoAtNet
- Audio Spectrogram Transformer

Start with a small CNN or ResNet-style model, then compare against CoAtNet later.

The acoustic model should output probabilities, not just one label:

```
k: 0.41
j: 0.33
h: 0.08
u: 0.05
...
```

These probabilities are needed by the fusion model.

Useful acoustic splits:

- same-user split
- cross-user split
- same-session split
- cross-session split

The same-user and same-session setting will be easiest. Cross-user and cross-session are more meaningful.

Phase 6: Extract Timing Features

For every typed sequence, compute rhythm features.

Basic timing features:

```
dwelt time = keyup_i - keydown_i
press-press latency = keydown_{i+1} - keydown_i
release-press latency = keydown_{i+1} - keyup_i
release-release latency = keyup_{i+1} - keyup_i
pause length
burst length
typing speed
```

Also compute keyboard-geometry features for each possible key transition:

```
same hand or different hand
```

same finger or different finger
physical key distance
row change
column change
left-to-right or right-to-left motion

Timing alone may not identify isolated keys very well, but it can help rank possible sequences.

Phase 7: Build A Timing-Only Baseline

Before combining timing with acoustics, test what timing can do by itself.

Possible timing-only tasks:

- identify the user
- detect word boundaries
- classify transition type
- distinguish phrase typing from random typing
- rank likely typed strings
- predict the next key group

Simple model choices:

- logistic regression
- random forest
- gradient boosted trees
- Hidden Markov Model

Sequence model choices:

- LSTM
- GRU
- Transformer

Start simple. A random forest or gradient boosted tree model is a good first baseline.

Phase 8: Build The Fusion Model

The fusion model combines acoustic and timing evidence.

A simple probabilistic formulation:

```
P(keys | audio, timing)
proportional to
product_i P(audio_i | key_i)
*
product_i P(delta_t_i | key_i, key_{i+1})
*
P(language_sequence)
```

Interpretation:

- $P(\text{audio}_i | \text{key}_i)$ comes from the acoustic classifier
- $P(\text{delta}_t_i | \text{key}_i, \text{key}_{i+1})$ comes from timing distributions
- $P(\text{language_sequence})$ is optional and should be evaluated separately

Use beam search for decoding:

```
for each keystroke:
    keep top N candidate sequences
    extend each sequence with possible next keys
    score each extension using acoustic + timing evidence
    keep the best N sequences
```

This gives top-k candidate strings instead of only one prediction.

Phase 9: Compare Four Systems

The main experiment should compare:

1. acoustic only
2. timing only
3. acoustic + timing
4. acoustic + timing + language model

Keep the language model separate. Otherwise, it becomes hard to tell whether timing helped or whether English statistics did all the work.

Important metrics:

- top-1 character accuracy
- top-5 character accuracy
- string recovery accuracy
- character error rate
- edit distance
- word accuracy
- top-k candidate rank
- synthetic password candidate rank

The most important result is:

Does acoustic + timing beat acoustic only?

Pay special attention to cases where the acoustic model is uncertain:

- nearby keys
- same-row neighbors
- same-hand transitions
- keys with similar spectrograms

Phase 10: Run Ablation Studies

Ablation means removing one component at a time.

Test the fusion system:

with and without dwell time
with and without flight time
with and without keyboard distance
with and without same-hand features
with and without language model
with and without acoustic confidence

Useful research questions:

- Does timing help more for fast typists?
- Does timing help more for adjacent-key confusions?
- Does timing help more for words than random strings?
- Does timing generalize across users?
- Does timing generalize across keyboards?
- Does timing help more when acoustic confidence is low?

Phase 11: Expand Gradually

Do not start with the hardest version.

Phase 1:

one user
one keyboard
one microphone
known prompts
oracle segmentation

Phase 2:

multiple users
same keyboard
same microphone
known prompts
automatic segmentation

Phase 3:

- multiple keyboards
- multiple users
- multiple microphone positions
- unknown prompts

Phase 4:

- Zoom or VoIP audio
- background noise
- natural typing
- unseen users
- unseen keyboards

Each phase should only add one or two sources of difficulty.

Phase 12: Suggested Repository Structure

Use a simple project layout:

```
keyboard-fusion/  
  data/  
    raw/  
    processed/  
    metadata/  
  notebooks/  
  src/  
    collection/  
    preprocessing/  
    segmentation/  
    acoustic/  
    timing/  
    fusion/  
    evaluation/  
  models/  
  reports/  
  configs/
```

Useful scripts:

```
collect_trials.py  
extract_keystrokes.py  
make_spectrograms.py  
train_acoustic.py  
extract_timing_features.py  
train_timing.py  
decode_fusion.py  
evaluate.py
```

Phase 13: Minimum Viable Version

The smallest useful version:

```
1 user
1 keyboard
1 microphone
36 keys
isolated key calibration
50 short phrases
audio + key timestamps
log-mel spectrogram CNN
timing feature extractor
beam-search fusion
compare acoustic-only vs acoustic + timing
```

This version is small enough to build quickly but meaningful enough to test the core hypothesis.

Phase 14: What A Successful Result Looks Like

A plausible first result might look like:

```
acoustic only: 82% character accuracy
timing only: weak exact-key accuracy, useful sequence signal
acoustic + timing: 86% to 89% character accuracy
acoustic + timing + language model: higher on English phrases
```

Even a small improvement is meaningful if it is consistent and well-controlled.

The strongest claim would be:

Timing information improves acoustic key recovery most when the acoustic classifier is uncertain between physically or acoustically similar keys.

That is a clean and believable research contribution.

Phase 15: Ethical And Defensive Framing

Use only consent-based data collection.

Do not:

- collect real passwords
- record people without permission
- test against third-party systems
- present the project as a universal keyboard decoder

Do:

- use synthetic prompts
- clearly describe limitations
- measure risk under controlled assumptions
- test mitigations

- explain defenses

Good defenses to evaluate:

- white noise
- fake keystroke injection
- keyboard dampening
- microphone distance
- VoIP noise suppression
- randomized typing behavior
- constant-rate event batching

The strongest final project is not only:

Can we recover keys?

It is:

How much information leaks, when does timing help, and what reduces the leakage?

Recommended Build Order

Follow this order:

1. Build typing data collector
2. Record small pilot dataset
3. Align audio with keydown timestamps
4. Extract keystroke clips using oracle segmentation
5. Train acoustic classifier
6. Extract timing features
7. Train timing-only baseline
8. Build beam-search fusion decoder
9. Compare acoustic-only vs acoustic + timing
10. Add automatic segmentation
11. Expand users and keyboards
12. Add language model as a separate experiment
13. Run ablations
14. Test defenses
15. Write report

Final Notes

The project is meaningful because the original acoustic-only setup treats every keystroke independently, while real typing is sequential. Timing may not identify an isolated key by itself, but it can help choose between possible key sequences.

The expected contribution is not:

timing replaces acoustics

The better contribution is:

timing improves acoustic recovery when acoustic predictions are ambiguous